

CSED312 Project 2

Final Report

팀 오우노

20230355 정지성

20230345 이성재

목차

- 1. Process Termination Messages**
- 2. Argument Passing**
- 3. System Calls – File Manipulation**
- 4. System Calls – User Process Manipulation**
- 5. Denying Writes to Executables**

1. Process Termination Messages

1.1. 목표

현재의 pintos 구현에는 process가 종료되어도 종료 메시지를 출력하지 않는다. 그래서 이 부분의 구현목표는 process가 종료되었을 때, 어떤 exit code를 가지고 종료했는지 출력해 주는 기능을 구현하는 것이다. 메시지의 형식은 다음을 따를 것이다.

```
printf("%s: exit(%d)\n", process_name, exit_code);
```

1.2. 구현방식

1.2.1. 자료구조

`struct thread`에 `int exit_status` 자료구조를 추가했다. Process가 종료되는 시점에 `exit_status`의 정보와 process의 이름을 이용해 메시지를 출력하면 된다.

1.2.2. 알고리즘

`printf()` 함수를 이용해 간단하게 구현할 수 있다. 중요한 것은 process가 언제 종료되느냐는 것이다. 크게 두 곳이 있는데, 바로 (우리 구현에서 exit system call을 핸들링하는) `sys_exit()` 함수와, `userprog/exception.c`의 `kill()` 함수 중 user exception에 의한 종료 구간이 있다. 여기에 위에 작성한 printf 문을 넣으면 된다.

1.3. 논의

1.3.1. Design report와 달라진 점

Design report에서는 이 메시지를 `process_exit()` 함수에서 부르기로 했다. 그런데 이렇게 되면 문제점이, `process_exit()`까지 exit code를 propagate해야 된다는 것이다. 그래서, `process_exit()`보다 조금 더 앞은 구간인 `sys_exit()`와 user exception의 구간에 따로따로 printf문을 넣어주었다. 두 경우 모두 결국 `process_exit()`을 도달하긴 하지만, 위의 이유로 따로따로 넣어주기로 했다.

1.3.2. 배운점

User program이 종료할 수 있는 다양한 방법에 대해 생각해보게 되는 문제였다고 생각한다.

1.3.3. 한계점

`sys_exit()`과 `exception.c`의 `kill()` 함수 내부에 완전히 동일한 코드가 조금 적혀 있다. 이를 하나의 함수로 통합해서 redundant code를 줄일 수 있었겠지만 그러지 않았다. `exception.c` 파일에 syscall handler 함수를 불러오는 것은 조금 이상하다고 생각했고, 또 막상 다른 곳에 함수를 정의해 사용하자니 어떤 파일에 함수를 정의해야 하는지, 또 각 파일의 header에 어떤 파일을 또 include해야 하는지 머리가 복잡해졌다. 그래서 그냥 같은 코드를 복사해서 붙여넣는 것으로 구현했는데, 이를 잘 생각해서 함수로 만들었다면 조금 더 깔끔한 구현이 됐을 것이다.

2. Argument Passing

2.1. 목표

기존의 Pintos 코드는 user program의 argument를 제대로 이해하지 못했다. Argument을 하나도 푸쉬하지 않고 stack pointer을 user memory의 맨 바닥에 놓은 상태에서 프로그램을 실행하기 때문에, 이 상태에서 argument를 읽으려고 하면 invalid memory access가 되어 exception이 일어난다. 따라서 이 부분의 구현목표는 command line string을 받으면 이를 whitespace에 맞게 parsing하고, x86 calling convention을 따라서 스택에 argument를 적절히 push하는 것을 목표로 한다.

2.2. 구현방식

2.2.1. 자료구조

크게 두 개의 function을 정의하고 사용했다.

1. `int argument_parse(char *cmd_line, char **argv)`

- string `cmd_line`을 받아, space를 기준으로 parsing해서 `argv` array에 저장하는 함수이다. 또한 parsing된 덩어리의 개수를 반환한다. (이를 `int argc`에 저장할 것이다.)

2. `void argument_push(void **esp, int argc, char **argv)`

- 포인터 `esp`와 parsing된 string array `argv`를 받아, x86 convention에 맞게 stack에 argument들을 push하는 기능을 하는 함수이다.

2.2.2. 알고리즘

각각의 함수는 다음과 같이 구현되었다.

1. `int argument_parse(char *cmd_line, char **argv)`

- `strtok_r()` 함수를 이용해 parsing한다.

2. `void argument_push(void **esp, int argc, char **argv)`

Push하고자 하는 데이터의 크기가 `n`일 때, push를 하는 기본적 원리는 `esp`를 `n`만큼 감소시킨 뒤 `esp`의 자리에 데이터를 넣는 것이다. 이 함수가 하는 기능은 5개의 step으로 나누어 설명한다. 각 step이 어디를 가리키는지는 source code에 주석으로 적혀 있다.

- (a) parsing된 각각의 string data 그 자체를 push한다. Null terminating을 쉽게 하기 위해 `strcpy` 함수를 이용한다. 이 때 step 3를 위해 각 string의 address도 bookkeeping해둔다.
- (b) Word align을 위해, `esp % 4 == 0`이 될 때까지 stack에 `uint8_t = 0`을 넣어 준다.
- (c) (a)에서 bookkeeping해 둔 각 argv의 주소를 push한다. 이 때 argv array를 null terminate하기 위해, `argv_addresses[argc] = NULL`도 push해 준다. push되는 각 element의 data type은 `char*`일 것이다.
- (d) Argv array 자체의 address를 push한다. 이의 data type은 `char**`일 것이다.
- (e) 마지막으로, return address를 0으로 push한다. 이의 data type은 `void*`이다.

2.3. 논의

2.3.1. Design report와 달라진 점

Design report를 적었을 당시에는 parsing을 따로 하지 않고, 대신 command line 전체 string을 우선 stack에 푸쉬한 뒤 whitespace가 나올 때마다 `\0`으로 바꿀 생각이었다. 하지만 코드를 짜 보니 이것에는 몇 가지 문제가 있었다.

- Whitespace가 두개 이상인 경우에는 위의 구현에서는 `\0`이 두 개 연달아 쓰이는 형태가 될 것이다. 하지만 더 나은 구현은 `\0`이 하나만 등장하는 것일 것이라고 생각했다. (사실 convention을 어긴 것은 아니라 동작에 문제는 없을 것이라고 생각된다)
- `start_process()` 함수에서는 `file_name`을 이용해 `thread_create()`를 실행하며, 이후 `file_name`으로 `load()` 함수를 실행한다. 그런데 이 `file_name`은 argument가 분리되지 않은 형태여서 parsing이 필요하다. 우리의 stack pushing 함수는 적어도 `load()` 코드 안 혹은 그 이후에 등장해야 하는데, parsing은 그보다 더 앞에서 필요하므로, parsing logic을 pushing logic에서 분리해서 사용해야 한다는 결론을 내렸다.

이러한 이유들로, parsing을 담당하는 `argument_parse()` 함수를 새로 정의해 사용하였다.

2.3.2. 배운점

x86의 calling convention을 직접 구현해 보면서 더 익숙해질 수 있었다. Stack pointer의 값을 직접적으로 수정하며 os를 더 low level한 관점에서 이해할 수 있었다.

2.3.3. 한계점

우리의 구현은 whitespace 중 ascii로 `0x20`인 space에 대해서만 parsing을 진행한다. `\t`나 `\v`같은 다른 whitespace에 대해서는 parsing을 진행하지 않는다. 이를 수정하는 방법은 매우 간단한데, 우리의 parsing 구현에서 `strtok_r()` 함수의 delimiter에 해당 whitespace character들을 추가해주면 된다.

3. System Calls – File Manipulation

3.1. 목표

3장과 4장에서는 system call을 handling하는 로직을 구현한다. 기존의 pintos 구현에서는 system call을 받으면 “system call!”을 출력하고 `thread_exit()`을 호출함으로써 system call을 ‘handling’하는데, 이번과 다음 장에서 handling logic을 구현하고, 이후 각각의 system call을 처리해줄 수 있는 함수를 구현하면 된다.

3.2. 구현방식

3.2.1. General System Call Handling Logic

System call이 발생할 때, system call handler의 스택에는 `arg3`, `arg2`, `arg1`, syscall number 순서로 데이터가 push된다. 각각의 `arg`는 word size, 즉 4바이트이다. (물론 syscall이 요구하는 `arg`의 개수에 따라 push되는 `arg`의 개수도 다를 것이다.) 따라서 스택의 각각의 데이터를 꺼내는 방법은 `((uint32_t *) f->esp + n)`를 통해 할 수 있음을 알 수 있다. (`n = 0`이면 syscall number, 이후로 `n`의 값은 `n`번째 `arg`를 꺼냄)

우리의 `syscall_handler()`는 먼저 위와 같은 방식으로 syscall number을 알아내고, 이를 case문으로 처리한다. 각각의 case에 맞는 `sys_something()`을 정의한 뒤, 위의 방식을 통해 `arg`들을 불러와 `sys_something()`에 넣고, 이 함수의 반환값을 `f->eax`에 넣어 주는 것을 골격으로 선택했다. 이렇게 하면 각각의 syscall handling은 `sys_something()` 함수를 작성하는 것으로 문제를 단순화할 수 있다.

3.2.2. 자료 구조: `fd_table`, `next_fd`, Helper Functions

파일을 다루기 위해서는 정수 `fd`와 파일 `struct file`을 매핑해주는 어떤 자료 구조가 필요하다. 우리는 이것을 `struct file_handle`을 원소로 하는 `fd_table` 리스트를 `struct thread`에 추가해줌으로써 달성했다.

```
struct file_handle {
    int fd;
    struct file *file;
    struct list_elem elem;
};
```

또한, 이 구조체를 편하게 다루기 위해 몇 가지 helper function을 정의했다.

- `lookup_handle_by_fd()`: `fd`를 받으면 그에 맞는 `struct file_handle`을 반환하는 함수이다. 찾지 못하면 `NULL`을 반환한다.
- `fd_to_file()`: `fd`를 받으면 그에 맞는 `struct file`을 반환하는 함수이다. 찾지 못하면 `NULL`을 반환한다.

파일을 새로 open하게 되면, `filesys_open()`이 반환해주는 `struct file`에 `next_fd`로 새롭고 고유한 `fd`값을 배정해 주어 `file_handle`을 만들어 주면 된다. `next_fd`는 배정 시마다 1 증가시킨다.

3.2.3. sys_create()

파일명과 초기 크기를 받아서, 해당 파일을 만들어 주는 함수이다. 받은 인자를 그대로 `filesys_create()`에 넣어 부르면 끝이다.

여기서 앞으로의 syscall에도 적용이 될 만한 내용을 몇 개 추가로 정리하겠다.

- 포인터를 받으면 거의 무조건 `validate_ptr()` 함수를 이용한다. 이는 나쁜 포인터를 쳐내는 함수인데, `ptr value ~ ptr value + 3`의 word단위로 그 값이 user vaddr에 있는지, 또 allocated된 page 내부에 있는지 검사하는 함수이다. 둘 중 하나라도 아니라면 `sys_exit()`를 불러서 프로그램을 종료시켜 버린다.
- `filesys/file.c`나 `filesys/filesys.c`의 함수를 사용할 때에는 `file_lock`을 꼭 사용해 주어야 한다. 이는 현재의 file system implementation에는 synchronization safe하지 않기 때문이다.

위의 두 내용은 앞으로의 system call handling function에도 자주 등장할텐데, 보고서에서는 반복하여 언급하지 않겠다.

3.2.4. sys_remove()

파일명을 받아, 해당 파일을 삭제해 주는 함수이다. 받은 인자를 그대로 `filesys_remove()`에 넣어 주면 된다.

3.2.5. sys_open()

파일명을 받아, 해당 파일을 열어 주는 함수이다. 우선 인자를 그대로 `filesys_open()`에 넘겨 주어 호출하면 `struct file`을 반환받는다. 하지만 process가 이 파일을 계속 접근하기 위해서는 앞서 정의한 `fd_table`에 `fd`와 함께 이 `struct file`을 묶는 entry를 추가해 주어야 한다. 함수에서 그 작업도 같이 해 주고 있다.

한 가지 중요한 것은, 우리는 `fd_table`의 entry를 만들 때 `malloc()`을 통해 메모리를 할당해 주고 있다. 이 때, process가 종료되거나 file을 닫게 되면 반드시 `free()`해주는 것을 잊지 않아야 한다! 그렇지 않으면 memory leak이 발생해 엄청난 자원 손해를 본다.

3.2.6. sys_filesize()

file descriptor을 받으면 그 파일의 크기를 반환해 주는 함수이다. 앞서 정의한 helper function `fd_to_file()`을 통해 `struct file`을 알아내고, 이를 `file_length()` 함수에 넣어 주면 된다. 이 때, `fd`가 0이나 1인 경우 이는 `stdin`이거나 `stdout`이므로 예외처리를 해준다.

3.2.7. sys_read()

`fd`와 `buffer`, `size`를 input으로 받는다. `fd`의 파일에서부터 `size`만큼 읽어 `buffer`에 저장해주어야 한다. `fd != 0`인 경우 `fd`에 해당하는 `struct file`을 찾은 뒤 나머지 input과 함께 그대로 `file_read()`함수에 넣어 주면 된다. `sys_read()`는 `fd == 0`인 경우 특정한 파일이 아닌 `stdin`에서부터 읽는다. 이는 `input_getc()` 함수를 이용해 읽어줄 수 있다.

3.2.8. sys_write()

`fd`와 `buffer`, `size`를 input으로 받는다. `fd`의 파일에서부터 `buffer`의 내용을 `size`만큼 읽어 `file`에 작성해주는 함수이다. `fd != 1`인 경우, `fd`에 해당하는 `struct file`을 찾은 뒤 나머지 input과 함께 그대로 `file_write()` 함수에 넣어 주면 된다. `sys_write()`는 `fd == 1`인 경우 특정한 파일이 아닌 `stdout`으로 write를 진행한다. 이는 `putbuf()` 함수를 이용해 출력해줄 수 있다.

3.2.9. sys_seek()

`fd`와 `position`을 받으면, `fd`의 cursor를 `position`으로 옮기면 되는 함수이다. `stdin`이나 `stdout`이면 `-1`을 반환하고, 이외의 경우 `fd` 인자에 해당하는 `struct file`을 찾아 `file_seek()`에 넣어 주면 된다.

3.2.10. sys_tell()

`fd`를 받으면, `fd`의 cursor 위치를 알려 주는 함수이다. `position`으로 옮기면 되는 함수이다. `stdin`이나 `stdout`이면 `-1`을 반환하고, 이외의 경우 `fd` 인자에 해당하는 `struct file`을 찾아 그대로 `file_tell()`에 넣어 주면 된다.

3.2.11. sys_close()

`fd`를 받으면, `fd`에 해당하는 파일을 닫아 주는 함수이다. `stdin`이나 `stdout`이면 일찍 반환하고, `fd`에 해당하는 `struct file`을 찾은 뒤 `file_close()`를 불러 주면 된다. 중요한 점은, file을 close하면 우리가 정의했던 `struct file_handle`에 해당하는 메모리를 free해주어야 하며, 우리의 `fd_table` 리스트에도 이 element를 제거해 주어야 한다는 것이다.

3.3. 논의

3.3.1. Design report와 달라진 점

가장 큰 차이는 `file_holder` 라는 구조체를 도입했다는 것이다. `fd`의 상한선을 설정하지 않기 위해서는 array같은 방식보다 이런 방식이 더 적합하다는 생각에 디자인의 방향을 이렇게 바꾸었다. 파일과 관련된 시스템 콜들을 구현할 때 디자인 레포트에서는 `fd_table`의 `fd`번째 인덱스로 불러오거나 `NULL`로 만드는 방식을 생각했었지만, `fd_table`을 linked list로 구현하며 많은 system call 함수들이 바꼈다.

3.3.2. 배운점

`tests/userprog/no-vm/multi-oom` 테스트 케이스를 해결하면서 memory leak을 방지하는 것에 대한 중요성을 알게 되었다. 할당된 메모리를 제때 free해 주지 않으면 생각보다 빠르게 os의 메모리가 꽉 차게 된다는 것을 알게 되었다.

3.3.3. 한계점

지금의 pintos에서의 file system은 file size가 고정되어 있고 파일 이름은 14자를 넘을 수 없는 등, 굉장히 한정적이다. 이는 CS50에서는 하지 않겠지만 project 4을 작업하면 해소될 수 있다고 한다.

4. System Calls – User Process Manipulation

4.1. 목표

이 부분에서는 User process를 관리할 수 있는 system call을 구현하는 것을 목표로 한다. 구현해야 하는 system call에는 `halt()`, `exit()`, `exec()`, `wait()`이 있다.

4.2. 구현방식

기본적으로 system call의 발생부터 아래의 각각의 `sys_something()` 함수까지 도달하는 방식, user에게 반환값을 돌려주는 방식은 3장에서 설명하였다. 따라서 아래에서는 각 syscall handler의 구현방식만 설명해도 충분할 것이다.

4.2.1. `sys_halt()`

Pintos에 이미 구현되어 있는 `shutdown_power_off()` 함수를 호출하면 된다.

4.2.2. `sys_exit()`

`thread_exit()`을 부르는 것이 기본적인 목적이다. 하지만, 필요에 따라 여러 가지 기능이 추가되었다. 아래에 그 기능을 나열해 놓았으며, 각각의 기능이 설명된 보고서의 위치도 같이 정리해 두었다.

- Process termination message 출력하기 (1장)
- 실행했던 binary의 executable file에 대해 `file_allow_write()` 부르기 (5장)
- Child processes의 `zombie_sema` 증가시키기 - 나중에 child가 terminate하게 되면, reap되기를 기다리지 않고 바로 죽을 수 있도록 해 준다. (4.2.3.)
- 자신의 `wait_sema` 증가시키고 `zombie_sema` 감소시키기. Wait하고 있던 parent를 깨우고, reap될 때까지 기다리는 기능을 해 준다. (4.2.3.)

추가로, 1.3.3.에서도 언급했지만 `userprog/exception.c`의 user exception 부분에서는 위에 적은 `sys_exit()`의 부가적 기능을 담은 코드가 그대로 담겨 있다.

`thread_exit()`는 이후 `process_exit()`을 불러 process와 관련되어 배정되었던 메모리를 모두 free해주고, 본인을 `THREAD_DYING`상태로 바꾸어 죽기를 기다린다.

4.2.3. `sys_exec()`과 `sys_wait()`

이 두 system call은 밀접한 관련이 있기 때문에 같이 다룬다.

4.2.3.1. 구현을 위해 `struct thread`에 추가한 data structures

```
/* For exec */
struct list child_list;      /* Parent-owned child list */
struct list_elem child_elem; /* Child-owned. */
struct semaphore load_sema;  /* Child-owned. Parent does down, child does up */
bool load_success;           /* Child-owned; load success status of child */

/* For wait */
int exit_status;             /* Exit status of the process */
struct semaphore wait_sema;  /* Child-owned. parent waits for child to terminate */
struct semaphore zombie_sema; /* Child-owned. child waits for parent to reap it */
```

위의 `exec()`과 `wait()` 시스템 콜 처리를 위해 `struct thread`에 추가한 data structure을 나타낸다.

각각에 대해 설명하면 다음과 같다.

- `child_list`: Parent가 관리하는 child list이다. List의 element는 child들에 대한 `struct thread`이다. `exec()`나 `wait()`는 parent가 child에게 무언가를 해주어야 하는 system call이기 때문에 이러한 list를 관리하는 것이 중요하다.
- `child_elem`: Child가 parent의 `child_list`에 속하기 위해 존재하는 data structure이다.
- `load_sema`와 `load_success`: Child가 가지고 있다. Parent가 `exec()`을 통해 child를 만들면, 그 child의 `load_sema`에 `sema_down()`을 하여 child의 loading이 끝날 때까지 기다린다. Child는 loading이 끝났을 때 `load_sema`에 `sema_up()`을 하여 parent를 깨운다.
- `wait_sema`: child가 가지고 있다. Parent가 child를 기다리고 싶으면, parent는 child의 `wait_sema`에 `sema_down()`을 한다. Child가 terminate할 때 `wait_sema`에 `sema_up()`을 하여 자고 있던 parent를 깨운다.
- `zombie_sema`: child가 가지고 있다. child가 terminate할 때 `zombie_sema`에 `sema_down()`을 해 준다. Parent가 child에 wait을 할 때 `zombie_sema`에 `sema_up()`을 해 준다. 이를 통해 기초적인 `wait()`을 구현할 수 있다.
또한, parent가 terminate할 때에는 자신의 모든 child에 대해 `zombie_sema`에 `sema_up()`을 해 준다. 이렇게 구현했을 때, 모든 process가 언젠가는 terminate한다는 가정을 가지고 있다면 모든 child가 terminate된 이후에 언젠가 free된다는 보장을 해줄 수 있다.

4.2.3.2. `child_list`의 처리

Parent가 관리하는 `child_list`는 코드의 여러 군데에서 수정이 이루어지기 때문에, 어디서 어느 기능을 수행하는지 확실히 정리한 후 코딩하는 것이 중요하다. 아래는 `child_list`가 수정되는 모든 곳을 정리하였다.

- append: Parent thread가 `process_execute()` 함수의 말미에서. Child의 load가 성공하든 안하든 일단 append하기 때문에, load가 실패했을 경우 다시 제거해야 할 수 있다.
- delete: Parent thread가. (1) `sys_exec()`의 말미에서 child의 load가 실패했을 경우, (2) `process_wait()`의 말미에서 child에 대한 wait이 끝난 경우.

4.2.3.3. `sys_exec()`의 설계

`sys_exec()`은 parent와 child간의 interplay가 이루어지기 때문에 코딩하기 까다롭다. Parent는 `sys_exec()` 함수에서, child는 `start_process()` 함수에서 `load()`을 부른 이후의 공간에 `sys_exec()`과 관련한 로직이 담겨 있다. 이를 참고하여, parent가 `exec()`을 통해 child를 생성했을 때 시간순으로 벌어지는 일들을 정리하면 다음과 같다.

1. (Parent) `process_execute(cmd_line)`를 이용해 child process를 생성한다.
 - 이 때 child process는 `process_execute()` → `thread_create()`를 통해 `start_process()`가 실행되는 thread이 만들어지고, `start_process()` → `load()`를 통해 binary를 process에 load 시도한다.
2. (Parent) 생성한 child의 `tid_t`에 해당하는 `struct thread`를 찾아 가, 거기에 있는 `load_sema`를 1 감소시킨다. 이를 통해 parent는 child가 load했을 때까지 기다리는 효과를 얻는다.
3. (Child) 앞서 정리한 function call chain을 통해 `start_process()` 안에서 `load()`를 실행한다. 이후 `load()`의 결과값을 자신의 `struct thread`의 `load_success`에 저장하고, 자신의 `load_sema`를 1 증가시킨다. 이를 통해 parent를 깨운다.
4. (Child) 만약 load가 실패하였다면, 자신의 `zombie_sema`를 1 감소시켜 reap되기를 기다린다.
5. (Parent) Child의 load가 성공했으면 `sys_exec()`의 반환값으로 child의 tid를 반환한다. 실패했다면, child를 `child_list`에서 제거한 후 child를 reap한다. `zombie_sema`를 1 증가시키는 방식으로 child를 reap한다.

4.2.3.4. `sys_wait()`의 설계

`sys_wait()` 또한 `sys_exec()`과 비슷하게 parent와 child이 상호작용을 하기 때문에 코딩하기 쉽지 않다. Parent는 `process_wait()` 함수에서, child는 `sys_exit()`과 `userprog/exception.c`의 user exception 부분의 공간에서 `sys_wait()`과 관련한 로직이 담겨 있다. (참고로, `sys_wait()`는 `process_wait()`를 그대로 부름으로써 구현된다.) 이를 참고하여, parent가 `wait()`을 통해 child를 기다리는 과정을 시간순으로 정리하면 다음과 같다.

1. (Parent) 만약 받은 `tid_t`가 자신의 `child_list`에 없으면 -1을 일찍 반환한다.
2. (Parent) child의 `struct thread`에 접근해 `wait_sema`를 1 감소시켜 child가 죽을 때까지 기다린다.
3. (Child) `sys_exit()` 내부 등의 죽기 직전 코드에서, 자신의 `wait_sema`를 1 증가시켜 parent를 깨운 후, 자신의 `zombie_sema`를 1 감소시켜 좀비 프로세스가 된다.
4. (Parent) 지금 실행되고 있다는 것은 child가 확실히 죽었다는 것이므로, child의 `struct thread`에서 exit code를 받아 온다. 이후 child를 자신의 `child_list`에서 제거시키고 `zombie_sema`를 증가시켜 reaping을 완료한다.
5. (Child) 이제 `thread_exit()`에 도달해서 없어지면 된다.

4.2.3.5. Reaping과 관련하여

현재 우리의 구현에서는 child가 reaping될 때까지 parent를 기다린다. 만약 infinitely running process가 없다고 가정하면, 여기서 parent가 terminate될 때 children의 `zombie_sema`를 각각 1 증가시키는 것만으로 child가 eventually reap될 것이라고 할 수 있다.

4.3. 논의

4.3.1. Design report와 달라진 점

처음 디자인할 때는 고려하지 못했는데, child process가 종료될 때 reaping process를 위해 `zombie_sema`를 사용했다. 부모가 종료될 때, 자신의 모든 자식들에 대한 정보를 정리하고, 자식들이 기다리고 있는 `zombie_sema`를 증가시켜 자식들이 스스로 메모리를 해제할 수 있게 해야 했는데, 이 부분을 처음엔 미처 생각하지 못했었다.

4.3.2. 배운점

semaphore같은 synchronization primitives를 적극적으로 이용해서 코딩하는 것은 처음인데, 생각보다 thread들 간의 순서 관계에 대해 고민할 것이 많아서 어려웠다. race condition bug이 발생했을 때 정확한 원인을 찾는 것이 자주 어려웠는데, 그래도 코드를 짜며 이런 부분에 대해 숙련도를 올린 것 같아서 좋다.

4.3.3. 한계점

Child가 reap될 것이라는 보장은 infinitely running process가 없다는 가정 아래에서 이루어졌다. 이는 충분히 합리적인 가정이지만 (대부분의 OS는 timeout 기능이 구현되어 있음), 이에 의존하지 않는 구현도 가능하다고 생각된다.

5. Denying Writes to Executables

5.1. 목표

우리가 구현한 syscall 중 read를 이용해 elf binary를 load할 수 있고, write를 이용해 elf binary의 내용물을 수정할 수 있다. 하지만 이미 read해서 load된 binary를 누군가가 write를 이용해 수정하게 되면 예상치 못한 많은 문제가 발생할 수 있기 때문에, load되어 실행되고 있는 executable에 대해서는 deny write를 해주어야 한다.

5.2. 구현방식

5.2.1. 자료구조

`struct thread`에 `struct file *executable`을 추가하였다. Thread가 executable을 load했을 때 `thread` 구조체 자체에서 그 file을 열어 `file_deny_write()`를 불러오고, exit할 때 `file_allow_write()`를 해주는 것이 아이디어이다.

5.2.2. 알고리즘

이제 저 두 함수를 적절한 곳에 호출만 하면 될 것이다.

- `file_deny_write()`은 `load()`함수에서 load success path의 꼬트머리에 추가하였다. 이 곳이 binary가 load 완료된 직후의 곳이라고 판단했기 때문이다.
- `file_allow_write()`은 process가 종료되는 두 위치에 불러오면 된다. `sys_exit()`과 `userprog/exception.c`의 `kill()` 함수 중 user exception에 의한 종료 부분에 부르면 되겠다.

5.3. 논의

5.3.1. Design report와 달라진 점

Design report에서는 `file_allow_write()`를 explicit하게 부르지 않겠다고 썼는데, 그냥 명시적으로 불러오는 것이 낫다고 판단했다. 나머지는 report과 비교해 크게 달라진 점이 없다.

5.3.2. 배운점

간단한 로직인데도, 생각보다 두 함수를 불러올 적절한 곳을 찾는 것이 어려웠다. 코딩을 할 때 무작정 코딩을 시작하지 말고, 충분한 계획을 세운 뒤 머릿속에서 말이 된 다음에 코딩을 하는 게 훨씬 성공률이 높다는 것을 다시금 깨달았다.

5.3.3. 한계점

구현한 기능 자체는 워낙 간단해서, 발생할 만한 한계점은 떠오르지 않는다.